Andy Garcia
Assignment 3
EECS 348
9/28
Google Gemini assisted with code comparisons

I compared code from ChatGPT and Claude on google chrome through their websites.
The prompt was copied generated by Gemini using instructions given by the assignment:

Here is a prompt you can feed into a generative AI to create the C++ program:

System Prompt / Meta Prompt:

You are an expert C++ programmer. Please write a complete, self-contained C++ program (`main.cpp`) that implements an email priority system for a CEO based on the following detailed specification:

1. Requirements & Data Structures

* **Data Types:** Define an `Email` struct to store `sender` (string), `subject` (string), and `date` (`MM-DD-YYYY` format, parsed into a format suitable for date comparison).
* **Priority Queue:** Implement a **MaxHeap** from scratch using a dynamic list/array (`Email* heap`). **Do not use any external or built-in heap/priority queue libraries.**
* **Priority Priority Logic:**
1. Primary key: **Sender Category Priority** (Higher rank wins):
* `Boss` (Priority 5)
* `Subordinate` (Priority 4)
* `Peer` (Priority 3)
* `ImportantPerson` (Priority 2)
* `OtherPerson` (Priority 1)

2. Secondary key: **Date** (Newest email wins if categories match).

* **Comparison Function:** Write a `compare(Email a, Email b)` helper function returning standard `-1`, `0`, or `1` values to maintain heap properties consistently during heapify up/down ops.

2. MaxHeap Functions to Implement

* `void initHeap(MaxHeap *hp)`
* `void insert(MaxHeap *hp, Email email)`
* `Email peek(MaxHeap *hp)` (Returns root element without removing)
* `Email extractMax(MaxHeap *hp)` (Removes and returns top email, re-heapifies down)
* standard `heapifyUp` and `heapifyDown` helper routines.

3. File Input & Commands

Read commands from `stdin` or a test file containing line-by-line commands:

* `EMAIL <sender category>, <subject line>, <date>`
* Parses category, subject line, and `MM-DD-YYYY` date string. Inserts the new email into the MaxHeap.

* `NEXT`
* Displays the next email at the top of the heap without removing it:
```
Next email:
Sender: <sender>
Subject: <subject>
Date: <date>

```

* If the queue is empty, handle gracefully (e.g., print a message or do nothing).

* `READ`
* Deletes/pops the highest priority email from the queue without printing its details.
* Must safely handle edge cases like calling `READ` on an empty queue.

* `COUNT`
* Displays the current number of unread emails:
```
There are <count> emails to read.

```

4. Code Constraints & Edge Cases

* Code must handle consecutive `NEXT` operations without mutating the heap state.
* Code must handle consecutive `READ` operations correctly (removing items without extra output).
* Ensure proper memory allocation and cleanup (free dynamically allocated memory upon program exit).
* Standard C libraries allowed: `<stdio.h>`, `<stdlib.h>`, `<string.h>`.

Write robust, well-commented C++ code that follows these exact rules.

Claude's Code

```
// main.cpp - CEO Email Priority System
// Uses a hand-built MaxHeap (no STL heap / priority_queue).
// Only <stdio.h>, <stdlib.h>, <string.h> are used.
//
// Commands (one per line, from stdin or a redirected file):
//   EMAIL <category>, <subject>, <MM-DD-YYYY>
//   NEXT   -> show top email without removing it
//   READ   -> remove top email silently
//   COUNT  -> print number of unread emails
//
// Build: g++ -std=c++11 -Wall -o email main.cpp
// Run:   ./email < commands.txt

#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>

#define MAX_SENDER  32
#define MAX_SUBJECT 256
#define MAX_DATE    16
#define MAX_LINE    1024

// ---------------------------------------------------------------
// Data types
// ---------------------------------------------------------------
struct Email {
    char sender[MAX_SENDER];    // category name, e.g. "Boss"
    char subject[MAX_SUBJECT];
    char date[MAX_DATE];        // original "MM-DD-YYYY" text (for printing)
    int  priority;              // 5 (Boss) .. 1 (OtherPerson)
    int  dateKey;               // YYYYMMDD -> plain int comparison works
};

struct MaxHeap {
    Email *heap;   // dynamic array of emails
    int size;      // number of emails currently stored
    int capacity;  // allocated slots
};

// ---------------------------------------------------------------
// Comparison
// ---------------------------------------------------------------

// Returns  1 if a has higher priority than b,
//         -1 if a has lower priority than b,
//          0 if they are equal.
// Primary key: sender category priority. Secondary key: newer date wins.
int compare(Email a, Email b) {
    if (a.priority != b.priority)
        return (a.priority > b.priority) ? 1 : -1;
    if (a.dateKey != b.dateKey)
        return (a.dateKey > b.dateKey) ? 1 : -1;
    return 0;
}

// ---------------------------------------------------------------
// Heap helpers
// ---------------------------------------------------------------
static void swapEmails(Email *x, Email *y) {
    Email tmp = *x;
    *x = *y;
    *y = tmp;
}

// Move the element at index i up until its parent is >= it.
void heapifyUp(MaxHeap *hp, int i) {
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (compare(hp->heap[i], hp->heap[parent]) > 0) {
```

```
            swapEmails(&hp->heap[i], &hp->heap[parent]);
            i = parent;
        } else {
            break;
        }
    }
}

// Move the element at index i down until both children are <= it.
void heapifyDown(MaxHeap *hp, int i) {
    while (1) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;

        if (left < hp->size && compare(hp->heap[left], hp->heap[largest]) > 0)
            largest = left;
        if (right < hp->size && compare(hp->heap[right], hp->heap[largest]) > 0)
            largest = right;

        if (largest == i) break;

        swapEmails(&hp->heap[i], &hp->heap[largest]);
        i = largest;
    }
}

// ---------------------------------------------------------------
// MaxHeap API
// ---------------------------------------------------------------
void initHeap(MaxHeap *hp) {
    hp->capacity = 8;
    hp->size = 0;
    hp->heap = (Email *)malloc(hp->capacity * sizeof(Email));
    if (hp->heap == NULL) {
        fprintf(stderr, "Out of memory\n");
        exit(1);
    }
}

void insert(MaxHeap *hp, Email email) {
    // Grow the array (double) when full
    if (hp->size == hp->capacity) {
        int newCap = hp->capacity * 2;
        Email *tmp = (Email *)realloc(hp->heap, newCap * sizeof(Email));
        if (tmp == NULL) {
            fprintf(stderr, "Out of memory\n");
            exit(1);
        }
        hp->heap = tmp;
        hp->capacity = newCap;
    }
    hp->heap[hp->size] = email;
    heapifyUp(hp, hp->size);
```

```
    hp->size++;
}

// Caller must make sure the heap is not empty.
Email peek(MaxHeap *hp) {
    return hp->heap[0];
}

// Caller must make sure the heap is not empty.
Email extractMax(MaxHeap *hp) {
    Email top = hp->heap[0];
    hp->size--;
    if (hp->size > 0) {
        hp->heap[0] = hp->heap[hp->size];  // move last to root
        heapifyDown(hp, 0);
    }
    return top;
}

void destroyHeap(MaxHeap *hp) {
    free(hp->heap);
    hp->heap = NULL;
    hp->size = 0;
    hp->capacity = 0;
}

// ---------------------------------------------------------------
// Parsing helpers
// ---------------------------------------------------------------

// Trim leading/trailing whitespace in place.
static void trim(char *s) {
    int len = (int)strlen(s);
    while (len > 0 && (s[len - 1] == ' ' || s[len - 1] == '\t' ||
                       s[len - 1] == '\n' || s[len - 1] == '\r'))
        s[--len] = '\0';
    int start = 0;
    while (s[start] == ' ' || s[start] == '\t') start++;
    if (start > 0) memmove(s, s + start, strlen(s + start) + 1);
}

// Map category name to priority; returns 0 if unknown.
static int categoryPriority(const char *cat) {
    if (strcmp(cat, "Boss") == 0)            return 5;
    if (strcmp(cat, "Subordinate") == 0)     return 4;
    if (strcmp(cat, "Peer") == 0)            return 3;
    if (strcmp(cat, "ImportantPerson") == 0) return 2;
    if (strcmp(cat, "OtherPerson") == 0)     return 1;
    return 0;
}

// Parse "MM-DD-YYYY" into YYYYMMDD. Returns 0 on failure.
static int parseDate(const char *s) {
    int m, d, y;
```

```
    if (sscanf(s, "%d-%d-%d", &m, &d, &y) != 3) return 0;
    if (m < 1 || m > 12 || d < 1 || d > 31 || y < 0) return 0;
    return y * 10000 + m * 100 + d;
}

// Parse the part of an EMAIL line after the "EMAIL" keyword.
// Format: <category>, <subject>, <date>
// Subject may contain commas, so we split on the FIRST and LAST comma.
// Returns 1 on success, 0 on malformed input.
static int parseEmail(const char *rest, Email *out) {
    char buf[MAX_LINE];
    strncpy(buf, rest, MAX_LINE - 1);
    buf[MAX_LINE - 1] = '\0';

    char *first = strchr(buf, ',');
    char *last = strrchr(buf, ',');
    if (first == NULL || last == NULL || first == last) return 0;

    *first = '\0';
    *last = '\0';
    char *category = buf;
    char *subject = first + 1;
    char *date = last + 1;

    trim(category);
    trim(subject);
    trim(date);

    int pr = categoryPriority(category);
    int key = parseDate(date);
    if (pr == 0 || key == 0) return 0;

    strncpy(out->sender, category, MAX_SENDER - 1);
    out->sender[MAX_SENDER - 1] = '\0';
    strncpy(out->subject, subject, MAX_SUBJECT - 1);
    out->subject[MAX_SUBJECT - 1] = '\0';
    strncpy(out->date, date, MAX_DATE - 1);
    out->date[MAX_DATE - 1] = '\0';
    out->priority = pr;
    out->dateKey = key;
    return 1;
}

// ---------------------------------------------------------------
// Main: read commands line by line
// ---------------------------------------------------------------
int main() {
    MaxHeap hp;
    initHeap(&hp);

    char line[MAX_LINE];
    while (fgets(line, sizeof(line), stdin) != NULL) {
        trim(line);
        if (line[0] == '\0') continue;  // skip blank lines
```

```
        if (strncmp(line, "EMAIL", 5) == 0 &&
            (line[5] == ' ' || line[5] == '\t')) {
            Email e;
            if (parseEmail(line + 5, &e)) {
                insert(&hp, e);
            } else {
                fprintf(stderr, "Invalid EMAIL command: %s\n", line);
            }
        } else if (strcmp(line, "NEXT") == 0) {
            // Read-only: peek never changes the heap.
            // Empty queue -> print nothing.
            if (hp.size > 0) {
                Email e = peek(&hp);
                printf("Next email:\n");
                printf("Sender: %s\n", e.sender);
                printf("Subject: %s\n", e.subject);
                printf("Date: %s\n\n", e.date);
            }
        } else if (strcmp(line, "READ") == 0) {
            // Silently pop; safe (no-op) on an empty queue.
            if (hp.size > 0) extractMax(&hp);
        } else if (strcmp(line, "COUNT") == 0) {
            printf("There are %d emails to read.\n\n", hp.size);
        } else {
            fprintf(stderr, "Unknown command: %s\n", line);
        }
    }

    destroyHeap(&hp);  // free dynamic memory before exit
    return 0;
}
```

ChatGPT's Code

```
#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>

using namespace std;

// ------------------------------------------------------------
// Email structure
// ------------------------------------------------------------
struct Email {
    string sender;
    string subject;
    string date;

    // Store the date as YYYYMMDD so dates can be compared easily.
    int dateValue;
};
```

```
// ------------------------------------------------------------
// MaxHeap structure
// ------------------------------------------------------------
struct MaxHeap {
    Email* heap;
    int size;
    int capacity;
};

// ------------------------------------------------------------
// Convert MM-DD-YYYY into YYYYMMDD
// This makes comparing dates much easier.
// Example:
// 09-28-2026 -> 20260928
// ------------------------------------------------------------
int parseDate(string date) {
    int month, day, year;
    char dash1, dash2;

    stringstream ss(date);

    ss >> month >> dash1 >> day >> dash2 >> year;

    return year * 10000 + month * 100 + day;
}

// ------------------------------------------------------------
// Get the priority associated with a sender category
// ------------------------------------------------------------
int getPriority(string sender) {
    if (sender == "Boss")
        return 5;
    else if (sender == "Subordinate")
        return 4;
    else if (sender == "Peer")
        return 3;
    else if (sender == "ImportantPerson")
        return 2;
    else
        return 1;
}

// ------------------------------------------------------------
// Compare two emails.
//
// Returns:
//   1  -> a has higher priority than b
//   0  -> a and b have equal priority
//  -1  -> a has lower priority than b
//
// Primary comparison:
//     Sender priority
//
```

```
// Secondary comparison:
//     Newer date
// ---------------------------------------------------------------
int compare(Email a, Email b) {

    int priorityA = getPriority(a.sender);
    int priorityB = getPriority(b.sender);

    // First compare sender priorities.
    if (priorityA > priorityB)
        return 1;

    if (priorityA < priorityB)
        return -1;

    // If sender priorities are equal,
    // compare the dates.
    if (a.dateValue > b.dateValue)
        return 1;

    if (a.dateValue < b.dateValue)
        return -1;

    // Completely equal priority.
    return 0;
}

// ---------------------------------------------------------------
// Initialize the heap
// ---------------------------------------------------------------
void initHeap(MaxHeap* hp) {

    hp->capacity = 10;
    hp->size = 0;

    hp->heap = new Email[hp->capacity];
}

// ---------------------------------------------------------------
// Resize the heap when it becomes full
// ---------------------------------------------------------------
void resizeHeap(MaxHeap* hp) {

    int newCapacity = hp->capacity * 2;

    Email* newHeap = new Email[newCapacity];

    // Copy existing emails.
    for (int i = 0; i < hp->size; i++) {
        newHeap[i] = hp->heap[i];
    }

    // Delete old array.
    delete[] hp->heap;
```

```
    // Point to the new array.
    hp->heap = newHeap;
    hp->capacity = newCapacity;
}

// ------------------------------------------------------------
// Heapify Up
//
// Used after inserting a new element.
//
// The new element starts at the bottom of the heap and moves
// upward until the MaxHeap property is restored.
// ------------------------------------------------------------
void heapifyUp(MaxHeap* hp, int index) {

    while (index > 0) {

        // Parent of index i is (i - 1) / 2.
        int parent = (index - 1) / 2;

        // If current email has higher priority than its parent,
        // swap them.
        if (compare(hp->heap[index], hp->heap[parent]) > 0) {

            Email temp = hp->heap[index];
            hp->heap[index] = hp->heap[parent];
            hp->heap[parent] = temp;

            // Continue from the parent's position.
            index = parent;
        }
        else {
            // Heap property is already satisfied.
            break;
        }
    }
}

// ------------------------------------------------------------
// Heapify Down
//
// Used after removing the root.
//
// The last element is moved to the root and then moved downward
// until the MaxHeap property is restored.
// ------------------------------------------------------------
void heapifyDown(MaxHeap* hp, int index) {

    while (true) {

        int left = 2 * index + 1;
        int right = 2 * index + 2;
```

```
        // Assume the current element is the largest.
        int largest = index;

        // Check left child.
        if (left < hp->size &&
            compare(hp->heap[left], hp->heap[largest]) > 0) {

            largest = left;
        }

        // Check right child.
        if (right < hp->size &&
            compare(hp->heap[right], hp->heap[largest]) > 0) {

            largest = right;
        }

        // If the current element is already the largest,
        // the heap is fixed.
        if (largest == index)
            break;

        // Otherwise swap with the larger child.
        Email temp = hp->heap[index];
        hp->heap[index] = hp->heap[largest];
        hp->heap[largest] = temp;

        // Continue from the new position.
        index = largest;
    }
}

// ------------------------------------------------------------
// Insert an email into the MaxHeap
// ------------------------------------------------------------
void insert(MaxHeap* hp, Email email) {

    // Resize if necessary.
    if (hp->size == hp->capacity) {
        resizeHeap(hp);
    }

    // Put the new email at the end.
    hp->heap[hp->size] = email;

    // Increase number of emails.
    hp->size++;

    // Restore MaxHeap property.
    heapifyUp(hp, hp->size - 1);
}

// ------------------------------------------------------------
// Peek
```

```
//
// Returns the highest-priority email without removing it.
// ------------------------------------------------------------
Email peek(MaxHeap* hp) {
    return hp->heap[0];
}

// ------------------------------------------------------------
// Extract Max
//
// Removes and returns the highest-priority email.
// ------------------------------------------------------------
Email extractMax(MaxHeap* hp) {

    Email maxEmail = hp->heap[0];

    // Move the last email to the root.
    hp->heap[0] = hp->heap[hp->size - 1];

    // Decrease size.
    hp->size--;

    // Restore MaxHeap property if elements remain.
    if (hp->size > 0) {
        heapifyDown(hp, 0);
    }

    return maxEmail;
}

// ------------------------------------------------------------
// Free dynamically allocated memory
// ------------------------------------------------------------
void destroyHeap(MaxHeap* hp) {

    delete[] hp->heap;

    hp->heap = nullptr;
    hp->size = 0;
    hp->capacity = 0;
}

// ------------------------------------------------------------
// Main
// ------------------------------------------------------------
int main() {

    MaxHeap hp;

    initHeap(&hp);

    string line;

    // Read commands one line at a time.
```

```
while (getline(cin, line)) {

    // ----------------------------------------------------
    // EMAIL command
    // ----------------------------------------------------
    if (line.rfind("EMAIL ", 0) == 0) {

        // Remove "EMAIL " from the beginning.
        string data = line.substr(6);

        string sender;
        string subject;
        string date;

        // Find the commas separating the fields.
        size_t firstComma = data.find(',');
        size_t secondComma = data.find(',', firstComma + 1);

        if (firstComma == string::npos ||
            secondComma == string::npos) {
            continue;
        }

        // Extract sender.
        sender = data.substr(0, firstComma);

        // Extract subject.
        subject = data.substr(
            firstComma + 1,
            secondComma - firstComma - 1
        );

        // Extract date.
        date = data.substr(secondComma + 1);

        // Remove leading spaces.
        if (!subject.empty() && subject[0] == ' ')
            subject.erase(0, 1);

        if (!date.empty() && date[0] == ' ')
            date.erase(0, 1);

        // Create Email object.
        Email email;

        email.sender = sender;
        email.subject = subject;
        email.date = date;
        email.dateValue = parseDate(date);

        // Insert into heap.
        insert(&hp, email);
    }
```

```
        // ------------------------------------------------------
        // NEXT command
        // ------------------------------------------------------
        else if (line == "NEXT") {

            if (hp.size > 0) {

                Email next = peek(&hp);

                cout << "Next email:" << endl;
                cout << "Sender: " << next.sender << endl;
                cout << "Subject: " << next.subject << endl;
                cout << "Date: " << next.date << endl;
                cout << endl;
            }
        }

        // ------------------------------------------------------
        // READ command
        // ------------------------------------------------------
        else if (line == "READ") {

            // Only remove an email if one exists.
            if (hp.size > 0) {
                extractMax(&hp);
            }
        }

        // ------------------------------------------------------
        // COUNT command
        // ------------------------------------------------------
        else if (line == "COUNT") {

            cout << "There are "
                 << hp.size
                 << " emails to read."
                 << endl;
        }
    }

    // Clean up dynamically allocated memory.
    destroyHeap(&hp);

    return 0;
}
```

Code Analysis:

Guided by Google Gemini

	Claude	ChatGPT
Correctness	- Checks for "ImportantPerson" and "OtherPerson" without spaces, but the specification requires handling categories with spaces (e.g., "Important Person" and "Other Person"). - Checks for "ImportantPerson" and "OtherPerson" without spaces, but the specification requires handling categories with spaces (e.g., "Important Person" and "Other Person").	- Checks for "ImportantPerson" instead of "Important Person", assigning all other categories (including "Other Person") a default priority of 1 - Only removes leading spaces from subject and date strings; it fails to trim trailing spaces or proper tokens correctly when extra spaces exist around commas.
Execution Time (same for both)	- Insertion (insert): $O(\log N)$ average/worst-case due to heapifyUp, with an amortized $O(1)$ dynamic array insertion. - Extraction (extractMax / READ): $O(\log N)$ due to heapifyDown. - Peek (NEXT) & Count (COUNT): $O(1)$ time complexity. - Comparison (compare): $O(1)$ string/integer comparison	- Insertion (insert): $O(\log N)$ average/worst-case due to heapifyUp, with an amortized $O(1)$ dynamic array insertion. - Extraction (extractMax / READ): $O(\log N)$ due to heapifyDown. - Peek (NEXT) & Count (COUNT): $O(1)$ time complexity. - Comparison (compare): $O(1)$ string/integer comparison

Space Complexity (exact same for both)	- Heap Storage: $O(N)$ space where N is the number of active emails stored in the heap. - Dynamic Array Growth: Both double the array capacity upon reaching limit ($O(N)$ space reallocation). - Auxiliary Space per Operation: $O(1)$ auxiliary space for operations. - Claude pre-allocates fixed character buffers per email node, leading to constant memory sizes per struct but potential waste for short strings	- Heap Storage: $O(N)$ space where N is the number of active emails stored in the heap. - Dynamic Array Growth: Both double the array capacity upon reaching limit ($O(N)$ space reallocation). - Auxiliary Space per Operation: $O(1)$ auxiliary space for operations. - ChatGPT uses dynamic C++ strings, storing characters on the heap proportional to string length.
Maintainability	Uses procedural C idioms (malloc, realloc, fixed character arrays like char[256], raw pointers). It lacks object-oriented encapsulation and requires manual buffer sizing.	Uses standard C++ abstractions (std::string, std::stringstream, cin/cout). Code readability is higher, but business logic (such as parsing and priority assignment) is loosely structured inside procedural helper functions rather than encapsulated within classes.

f. Code Selection : ChatGPT

I selected ChatGPT's code because it uses an Object-Oriented C++ design which is more maintainable than Claude's C style procedural code. ChatGPT also had a better format on its output for the program.

g. Code revisions:

- Transformed Email and MaxHeap structs into robust C++ class structures with private member variables and public methods (insert, extractMax, peek, size, isEmpty).
- Updated category parsing to correctly match "Important Person" and "Other Person" with exact space formatting as defined in the specification.
- Standardized terminal output for NEXT and COUNT commands to match the sample test file requirements exactly.Standardized terminal output for NEXT and COUNT commands

to match the sample test file requirements exactly.Standardized terminal output for NEXT and COUNT commands to match the sample test file requirements exactly.

- Revised all comments to be on every line and thoroughly explain the code that is happening inside each block.
- a455g434@engr1005-05:~/eecs348/Fall2026_EECS348_Assignment_3/Andy_Garcia_Assignment_3$ git add .
- a455g434@engr1005-05:~/eecs348/Fall2026_EECS348_Assignment_3/Andy_Garcia_Assignment_3$ git commit -m "Added main.pp